RISC-V Avionics Demo

Architecture, evidence and viva handbook

Prepared for GN11 | Bandaru Rohith Datta and teammates | 4 November 2026

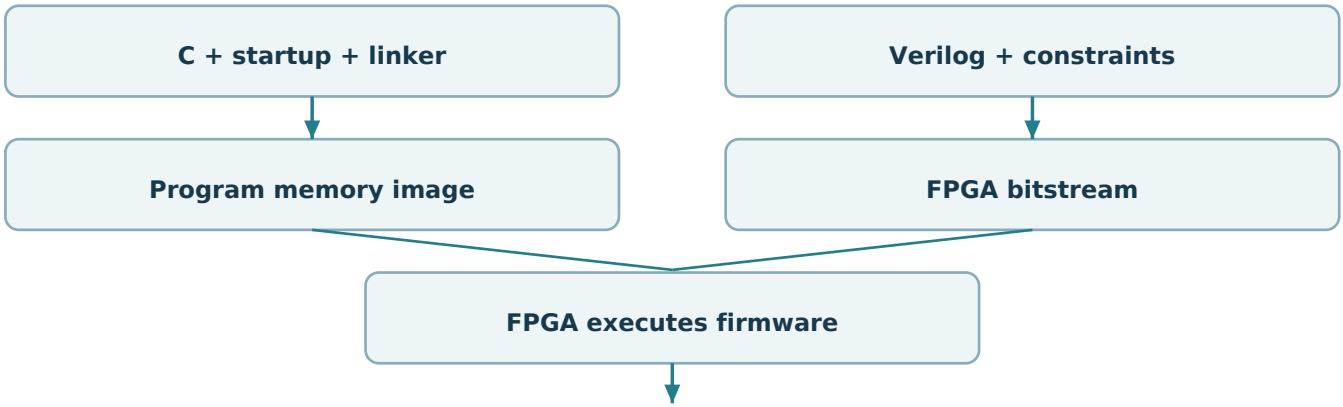
This guide describes the delivered processor_demo_timing_optimized.zip source and the supplied 40 MHz Vivado build log. It distinguishes what the code actually does from what comments, the original master prompt, or a preview image suggest. Design trade-offs below are engineering explanations of the implementation, not claims that the team benchmarked every alternative.

Verdict: the scoped regression passes, firmware rebuilds byte-identically, and fresh board simulations at 40 and 50 MHz each complete 100 packets and 57 self-checks with zero host failures. This audit fixes SRA/SRAI signedness, four host-validation gaps, misleading Zbb activity decoding and a D-cache stall label. The supplied earlier 40 MHz Vivado run found no negative setup/hold paths, but the corrected RTL requires a new implementation and timing run. No physical FPGA test was possible. Remaining ISA and observation limitations are documented; these tests do not establish full RV32IMF compliance or freedom from all bugs.

How to study this guide

Read sections 1-3 together first. Assign sections 4-11 to feature owners, then have everyone read sections 12-17: the demo, host verification, evidence and limitations are shared responsibilities. Use section 18 for a rehearsal and the file catalogue to locate the implementation. Whenever explaining a result, state whether it came from live hardware, RTL simulation, host arithmetic or a fixed expected test vector.

1. The system in one minute



This is a 32-bit, single-issue, in-order, five-stage processor implemented as FPGA logic. It executes a bare-metal RISC-V program from on-chip memory. The program generates a repeatable sensor-like input stream, performs byte manipulation and a floating-point exponential moving average (EMA), runs diagnostic checks, and sends data to a Python dashboard over UART. There is no operating system and no real IMU sensor in this demo.

The laptop does not execute the RISC-V firmware to produce the FPGA result. Its CPU runs Python, receives the FPGA's result bits, independently calculates selected reference results, and draws the interface. Vivado turns Verilog into a bitstream; the cross-compiler turns C and assembly into the program that the implemented processor executes. These are separate compilation paths.

Layer	Actual entry point	Responsibility
FPGA board	board/nexys_a7_top.v	Clock/reset, switches, RAM, core, UART, display, completion
Processor wrapper	integration/modules/core/rv32imf_core.v	Exposes memory, MMIO and observation interfaces
Processor internals	integration/modules/core/pipeline_5stage.v	Connects stages, caches, hazards, predictor and counters
Firmware startup	firmware/start.S	Establishes gp/sp, clears BSS, calls main
Firmware workload	firmware/avionics_demo.c	Self-tests, input construction, EMA, modes, UART records
Host application	dashboard.py	Serial reception, selected checks, plots, gauges and terminal

Viva answer: "We implemented the processor datapath and control in RTL. Our C program is the workload running on it. Python is the observer and reference checker. We have a deterministic demonstration rather than a live aircraft sensor system."

2. Exact baseline configuration

Property	Implemented value or qualification
FPGA target	Nexys A7-100T, xc7a100tcsg324-1
Board clock input	100 MHz; 10 ns XDC period
Current successful build	SYSTEM_CLOCK_HZ=40000000, supplied through build script
Source default	50 MHz remains the default; always select 40 explicitly when rebuilding
Supported script frequencies	50, 40, 31.25 and 25 MHz
Pipeline	IF, ID, EX, MEM, WB; single issue, in order
Instruction size	32 bits; no compressed-instruction execution path
Integer registers	x0-x31, 32-bit; x0 is hardwired zero
Floating registers	f0-f31, 32-bit; f0 is writable
Internal register identifiers	Six bits: bank bit plus five-bit register number
Unified backing RAM	16,384 x 32-bit words = 65,536 bytes
RAM addresses	0x00000000-0x0000FFFF; reset fetch PC is zero
I-cache / D-cache	Each 2 KiB payload, direct mapped, 128 lines, 16 bytes per line
BTB	32 entries, direct mapped, valid + 25-bit tag + 32-bit target
UART	TX only, nominal 115200 baud, 8 data bits, no parity, one stop bit
Completion	100 packet records after startup checks; sentinel then hardware clock stop

The 2 KiB cache capacity excludes tags and valid bits. Per cache, metadata is 128 x (21+1) = 2,816 bits, in addition to 16,384 data bits. The BTB stores 32 x (1+25+32) = 1,856 logical bits. These counts describe RTL storage, not the final LUT/BRAM utilization. Use the implemented utilization report for actual FPGA resources.

There are separate instruction and data caches, but they share one backing memory port. This resembles a modified Harvard organization: separate near-core paths, unified program/data storage. Instruction fetch and a D-cache hit can occur concurrently; simultaneous refills contend at the arbiter.

3. From source files to a visible result

3.1 Program build

The Makefile uses -march=rv32imf_zicsr_zbb, -mabi=ilp32f and -O1. The architecture option permits the compiler to emit these instructions; it does not prove that every instruction in those extensions is implemented. The actual binary must remain within the implemented subset.

- -ffreestanding, -nostdlib and -nostartfiles remove dependence on a host operating system and normal C runtime.
- -ffp-contract=off prevents contracting separate multiplies and additions into fused multiply-add, which the processor does not implement.
- -fno-fast-math preserves the intended numerical ordering and exceptional-value behavior.
- The linker script places code, constants, initialized data and BSS into 64 KiB RAM and reserves at least 4 KiB below the top for stack space.

The output chain is C + start.S + link.ld -> avionics_demo.elf -> avionics_demo.bin -> avionics_demo.mem. ELF contains structured executable information. The binary contains loaded program/data bytes. bin_to_mem.py converts each little-endian group of four bytes to one hexadecimal 32-bit word and pads the memory image to 64 KiB. The .dis file is a disassembly for checking what instructions were really emitted.

At FPGA configuration, $readmemh initializes the backing RAM from avionics_demo.mem. It does not load the I-cache directly. Cache valid bits reset to zero, so the first instruction fetch still misses and refills from that RAM. At boot, start.S initializes gp and sp, zeros BSS, then calls main. There is no loader downloading C over the UART.

3.2 Hardware build

build_vivado.tcl adds the RTL, board files, memory image and selected XDC, sets the top and clock generic, synthesizes, places and routes, writes reports and rejects negative setup/hold paths. It intentionally stops before bitstream generation. Open the completed .xpr and generate the bitstream afterward. Rebuilding the firmware requires updating the memory image in a new hardware build before assuming the FPGA is running the new program.

3.3 Data journey

1. Firmware constructs 100 deterministic 32-bit sensor-like words in RAM.

2. Loads pass through D-cache; rev8 transforms byte order; max clamps signed-negative bit patterns to zero; cpop counts one bits.

3. Floating arithmetic calculates the EMA result in the FPGA's FPU.

4. Firmware reads hardware CSRs and emits an ASCII CSV line using MMIO writes.

5. Separately, RTL captures pipeline snapshots, including events, WB state and cache counts.

6. The trace transport serializes complete lines while pausing the processor when necessary.

7. Python assembles serial bytes into lines, parses values, checks selected results and updates the dashboard.

This observation mechanism changes wall-clock execution time. It is not a passive high-bandwidth trace collector running alongside an uninterrupted 40 MHz program.

4. Feature 1: Five-stage pipeline and data hazards

4.1 What each stage does

Stage	Main file	Main operations
IF	fetch_stage.v	Holds PC, selects PC+4 or predicted target or EX recovery target; requests I-cache
ID	decode_stage.v	Decodes opcode/funct fields, reads register banks, constructs immediates and usage flags
EX	execute_stage.v	Selects forwarded operands; ALU, multiplier, divider, FPU, branch target/condition and CSR result
MEM	memory_stage.v plus pipeline wiring	Formats stores, decodes MMIO, accesses D-cache or peripheral data
WB	writeback_stage.v	Selects load/result data, sign/zero-extends subword loads and returns register write data

Each boundary has a register file: if_id_reg.v, id_ex_reg.v, ex_mem_reg.v and mem_wb_reg.v. They carry data, control, register identifiers and a valid bit. A stage label alone does not imply that stage always advances every cycle.

A bubble has valid=0 and inactive controls. A stall preserves selected stage contents. A flush discards wrong-path or unusable work by injecting a bubble. The registers prioritize reset, then flush, then normal transfer when not stalled. Clearing control signals is essential: an invalid instruction must not accidentally store to memory or modify a register.

4.2 Register banking

decode_stage.v physically declares regs[1:63]. Identifier zero represents x0 and is read as zero. Identifiers 1-31 are integer registers; 32-63 are floating registers. Thus x10 and f10 are different destinations even though both use architectural index ten. f0 is identifier 32, so it must not be suppressed by a destination-zero check.

Both register reads are combinational; writes occur on the clock edge. A WB-to-ID bypass returns the new WB value when the requested source matches the current write. The six-bit identifiers propagate through hazard and pipeline registers, preventing false x/f dependencies.

4.3 Forwarding and the load-use exception

hazard_unit.v gives EX/MEM forwarding priority over MEM/WB forwarding. This chooses the youngest older producer when both match. Forwarding affects arithmetic inputs, branch comparisons and store data. A direct ALU dependency can therefore execute without waiting for the register-file write.

Example: add x5,x1,x2 followed by sub x6,x5,x3. When sub reaches EX, add's result is in EX/MEM and is selected by the forwarding mux. A load is different: the EX result is its address, not its loaded data. For lw x5,0(x1) followed by add x6,x5,x2, the interlock freezes IF/ID and PC, injects an ID/EX bubble, lets the load advance, then uses WB forwarding. The dependency costs one bubble when the load otherwise hits; cache or arbitration waits add separately.

For a held ID/EX instruction, id_ex_reg.v also snoops WB and updates matching saved operand values. Otherwise a temporary WB forwarding opportunity could disappear before the stalled instruction resumes.

4.4 Stall priority

Cause	Held stages / response	Why
D-cache stall	Freeze through EX/MEM; bubble MEM/WB	Preserve pending memory access and avoid duplicate retirement
FPU/divider busy	Hold front end and ID/EX; bubble EX/MEM	Keep one operation active and prevent premature result transfer
Load-use	Hold PC and IF/ID; bubble ID/EX	Allow producer load to move forward
I-cache stall	Stop fetching new valid instructions; older instructions can drain	An instruction miss need not freeze all older work
Accepted misprediction	Redirect PC and flush IF/ID plus ID/EX	Remove younger wrong-path instructions

Why five stages? It divides logic into recognizable pieces and permits overlap with manageable control. A single-cycle CPU has simpler control but a long clock period; a deeper pipeline can raise frequency but increases bypass complexity and branch recovery cost. This design favors educational visibility and moderate FPGA complexity. It is not out-of-order: there is no rename table, reservation station or reorder buffer.

Viva check: latency is the time one instruction takes to traverse execution; throughput is how frequently results can be accepted. Ideal single-issue throughput is at most one retired instruction per active core cycle. Five stages do not mean five instructions retire per cycle.

5. Feature 2: Integer multiplication

multiplier_unit.v implements MUL, MULH, MULHSU and MULHU. MUL returns the lower 32 product bits; the other operations return the upper 32 with different signed interpretations. The low product bits are identical modulo 2^32 for signed and unsigned interpretations.

The board ties use_dsp_i to one, so the live demo selects multiplier_dsp.v. The DSP-oriented implementation extends each operand to 33 bits: sign-extension for signed operands, zero-extension for unsigned operands. A signed 33x33 multiplication can then cover all operand combinations; the low 64 bits form the architectural product. The attribute requests DSP inference. Comments about an exact DSP budget are not a substitute for the utilization report.

booth_radix4_multiplier.v is an alternative combinational implementation. It forms 17 overlapping three-bit recoding groups with factors 0, +1, -1, +2 or -2, shifts the partial products and adds them. It is present in source, but the board's constant selection permits synthesis to remove the unused alternative. There is no physical switch selecting Booth in the delivered board wiring, despite an older comment mentioning SW3. The USE_DSP parameter is also not the actual mux selector; use_dsp_sel is.

Instruction	Example	Expected result
MUL	10 x 20	0x000000C8
MULH	-1 x -1, high half	0x00000000
MULHU	0xFFFFFFFF x 0xFFFFFFFF, high half	0xFFFFFFFE
MULHSU	signed -1 x unsigned 0xFFFFFFFF, high half	0xFFFFFFFF

The unit is combinational in EX and mul_busy is always zero. "Single-cycle" means the complete multiplication path must fit inside one EX cycle; it does not mean zero physical delay. DSP resources are attractive on this FPGA because they provide dedicated arithmetic hardware. A sequential multiplier would save combinational resources but add stalls. A deeply pipelined multiplier would improve throughput only if the surrounding control could accept overlapping operations.

6. Feature 3: Integer division and remainder

divider_unit.v uses restoring-style long division. It operates on unsigned magnitudes, shifts the combined remainder/quotient, compares the trial remainder with the divisor, conditionally subtracts, and appends one quotient bit. Signed operations restore quotient sign from dividend XOR divisor sign and remainder sign from the dividend.

The first quotient step is combinational in IDLE; subsequent steps reuse the arithmetic in CALC, with the final step exposed on the completion cycle. Normal division occupies 32 EX cycles, typically 31 busy cycles plus the final service cycle. Divide-by-zero and signed-overflow corner cases bypass the iteration and complete in one EX cycle.

Operation	Required/result behavior in this implementation
DIV or DIVU by zero	Quotient 0xFFFFFFFF
REM or REMU by zero	Original dividend
signed INT_MIN / -1	Quotient 0x80000000
signed INT_MIN % -1	Remainder zero
signed ordinary division	Truncates toward zero; remainder follows dividend sign

These corner cases return values rather than causing arithmetic traps. LED6 combines the custom integer divide-zero observation with the FPU DZ event; integer divide-zero does not set floating-point fflags.DZ.

Why iterative division? A fully combinational divider would put many dependent compare/subtract steps on one timing path. Reusing one step reduces area and per-cycle logic depth, at the cost of waiting 32 cycles. The processor stalls while it runs, so an independent younger instruction does not bypass it.

7. Feature 4: Floating-point execution

7.1 Representation and supported scope

A binary32 word has sign bit 31, eight exponent bits 30:23 and 23 fraction bits 22:0. For normal numbers, value = (-1)^sign x (1 + fraction/2^23) x 2^(exponent-127). For subnormals, exponent is zero, the hidden bit is zero, and the scale is 2^-126. All-zero magnitude is signed zero. Exponent 255 with zero fraction is infinity; with nonzero fraction it is NaN.

Implemented arithmetic: FADD.S, FSUB.S, FMUL.S and FDIV.S. Additional execution-stage support includes FCVT.S.W/WU, FMV.W.X, FMV.X.W and sign injection; memory stages support FLW/FSW through the floating register bank. Missing full-F features include fused operations, square root, floating comparisons, floating min/max, FCLASS and float-to-integer conversion. Therefore use "single-precision FPU supporting the demo operations" rather than "complete RV32F compliance."

7.2 Internal operation schedule

Instruction	EX occupancy	Work sequence
FADD.S / FSUB.S	5 cycles	Capture/classify/align; add or subtract; normalize; round/pack; complete
FMUL.S	4 cycles	Capture operands; 24x24 significand multiply; normalize/round/pack; complete
FDIV.S	15 cycles	Capture/normalize; 13 calculation cycles producing two quotient bits each, final cycle also packs; complete

These internal registers do not make the unit throughput-pipelined. The FSM accepts one operation and holds later instructions until completion. "Unpipelined FPU" in the project means no overlapping independent operations, not an absence of registers inside the unit.

7.3 Addition and subtraction

Compare effective exponents/magnitudes, choose the larger magnitude, right-shift the smaller significand to align exponents and combine all discarded low bits into sticky. Add for equal effective signs or subtract otherwise. Normalize a carry by shifting right; normalize cancellation by counting leading zeros and shifting left. Then round and pack. FSUB flips B's effective sign before the addition machinery.

7.4 Multiplication and division

FMUL XORs signs, combines effective exponents, multiplies 24-bit significands into 48 bits and uses pack_rne to normalize and round, including tiny results. FDIV normalizes subnormal inputs, divides significands using two restoring steps per calculation cycle, retains quotient and remainder information, then packs a binary32 result. A nonzero residual remainder contributes sticky information.

The timing rewrite replaced a long serial leading-bit search with a binary search, narrowed exponent arithmetic and used a 25-bit rounding adder. The old and new pack functions matched on 20,000 tested magnitude/exponent combinations. This proves equivalence for those cases, not that either function is universally IEEE-correct.

7.5 Rounding and exceptions

For RNE, increment the retained significand when G AND (R OR S OR retained_LSB) is true. G is the first discarded bit, R the next, and S is the OR of all remaining discarded bits. At an exact halfway point, choose the neighbor whose last retained bit is even.

Example: 1.0 + 2^-24 is halfway between 1.0 and its next binary32 value; RNE returns 1.0. Starting at 0x3F800001 instead gives a halfway result that rounds up to 0x3F800002. A result can be numerically unchanged by rounding while NX still records inexactness.

fflags bit	Name	Meaning / example
4	NV	Invalid, such as 0/0, infinity-infinity, 0 x infinity
3	DZ	Finite nonzero floating value divided by zero
2	OF	Overflow beyond finite representable range
1	UF	Tiny inexact result under the implemented policy
0	NX	Rounded result differs from exact result

The canonical NaN is 0x7FC00000. Quiet NaN propagation and signaling NaN invalid detection are distinct. Exact subnormals must not automatically assert underflow: minimum normal multiplied by 0.5 gives exact 0x00400000 with no UF; the minimum subnormal multiplied by 0.5 rounds to zero and sets UF+NX.

Arithmetic exception events appear at FPU completion and are ORed into accrued CSR flags until cleared. fflags, frm and fcsr support read/replace/set/clear, but that storage does not imply complete rounding support. FMUL/FDIV ultimately use the RNE-only packer; the add path has mode-aware code but was not comprehensively validated outside RNE. FCVT.S.W/WU returns a rounded value through a separate combinational function and does not produce conversion NX flags. Keep the demonstrated program in RNE.

Why this design? Multi-cycle arithmetic reduces the amount of logic required in one cycle and exposes a clear handshake to the hazard unit. A full pipelined FPU would require more registers and control for concurrent operations; vendor IP might offer stronger compliance/performance but would hide much of the educational implementation. No measured power or area comparison against such alternatives was performed.

8. Features 5 and 6: L1 caches and memory arbitration

8.1 Organization and addressing

Both caches are direct mapped, with 128 lines of four 32-bit words. Address fields are tag[31:11], index[10:4], word offset[3:2] and byte offset[1:0]. Index selects the only possible line; valid plus tag comparison decides a hit; word offset selects the word.

Example: addresses 0x00001000 and 0x00001800 differ by 0x800 = 2 KiB. They have the same index but different tags, so alternate accesses conflict. Addresses 0x1000, 0x1004, 0x1008 and 0x100C share one line, so one refill supplies all four.

The data arrays explicitly request distributed RAM, supporting combinational hit reads. Old comments calling them four BRAM banks are inaccurate. The backing RAM separately requests block RAM and uses a synchronous read. Actual resource inference is a synthesis result.

Direct mapping needs one tag comparison and no replacement-policy state. A two-way cache could reduce conflicts but would add a second way, comparison/mux logic and replacement management. Four-word lines demonstrate spatial locality without a long refill burst. Larger lines can reduce some misses but increase transfer cost and pollution.

8.2 Nine stalls, ten cycles of occupancy

The corrected contract is exactly nine stall cycles plus a service cycle for a cold refill when memory requests are granted without extra waiting. It applies to I-cache and D-cache read misses and the modeled bypass accesses. At 40 MHz this is 225 ns of stall plus 25 ns service, or 250 ns active-clock occupancy. UART clock pauses can make the elapsed wall time much longer.

Relative cycle	Refill activity with immediate grants
1	IDLE detects miss; captures address and bypass mode; stall asserted
2-3	REFILL counter 0-1; modeled delay
4	Counter 2 issues word 0 request
5	Counter 3 captures word 0, issues word 1
6	Counter 4 captures word 1, issues word 2
7	Counter 5 captures word 2, issues word 3
8	Counter 6 captures word 3
9	Counter 7 completes line metadata or transitions to bypass release
10	Service/release: CPU accepts the requested word with stall low

The on-chip RAM itself is not physically a nine-cycle DRAM. The cache FSM intentionally models a slower backing-memory service so the cache experiment is visible. Four independent bypassed word accesses therefore have a baseline 4 x 9 = 36 stall cycles. There is no firmware multiplication that fabricates CSR values. Real counters are read. Arbitration can extend a refill, and a flush can cancel an instruction fetch.

8.3 D-cache stores

The D-cache is write-through and no-write-allocate. A cached store hit updates the relevant bytes and writes backing memory. A cached store miss writes backing memory without allocating a line. With an available grant these stores need no refill wait, even though a store miss increments a miss counter. Therefore D-cache misses x 9 is not generally equal to total D-cache stalls.

Bypassed stores use a separate delayed STORE state. Address, data and byte strobes are latched; the memory write occurs once at completion and the indexed cached line is invalidated. The saved payload is not a general write buffer that permits many stores to retire independently. Despite the header wording, there is no multi-entry write queue or dirty write-back policy.

Byte strobes permit SB, SH and SW. Loads are formatted later at WB for LB/LBU/LH/LHU/LW. Misaligned multi-byte accesses are not split across words and there is no complete alignment-trap mechanism; use aligned halfword/word accesses in the demo.

8.4 I-cache and branches

I-cache hits supply a word without an added miss stall. A misprediction flush cancels the active refill FSM; it does not invalidate every previously cached line. Bypass mode never treats a stored tag as a hit. The current firmware does not modify executable instructions; do not claim self-modifying-code coherence or FENCE.I support.

8.5 Shared bus

mem_bus_arbiter.v selects D-cache first when both requests arrive while idle. Once a side owns an active burst, it keeps ownership until its requests finish. A later D-cache request does not interrupt an ongoing I-cache burst. Response-valid tracking associates synchronous RAM data with granted requests. D-priority is useful because a blocked older memory instruction holds much of the pipeline, but strict priority is not a formal fairness guarantee against arbitrary sustained traffic.

Hardware DREFILL reports D-cache progress 1-9, and a grant wait can repeat a progress value. The UART does not export an equivalent IREFILL progress field. I-cache stalls are visible as events, but a claimed explicit "I-cache Cycle 1/9 ... 9/9" text sequence is not implemented.

9. Feature 7: Branch target buffer and recovery

The BTB has 32 direct-mapped entries indexed by PC[6:2], with tag PC[31:7]. A valid tag hit predicts taken and supplies the stored target; a miss predicts sequential PC+4. An accepted taken branch/jump writes its target and valid bit. A not-taken update invalidates the indexed entry.

This is a learned taken-target policy, similar to a simple one-bit last-outcome behavior with target storage. It does not examine the sign of the branch offset to implement a backward-taken/forward-not-taken rule. Forward taken branches can be learned too. Calling it purely static backward prediction is misleading.

Example: for a loop with a repeated taken branch, the first encounter normally misses the BTB and needs recovery. Subsequent encounters can hit and fetch the loop target without a prediction bubble. At loop exit, the stored taken prediction is wrong, so EX redirects to PC+4 and flushes the two younger slots. This is a two-slot squash; cache misses or other stalls can add to elapsed recovery time.

execute_stage.v checks both direction disagreement and wrong target for a taken prediction. JAL/JALR participate. There is no return-address stack; the BTB is the only target predictor. Code addresses separated by 128 bytes can alias in the BTB index, although the tag prevents treating a different PC as the same branch.

Why not a two-bit saturating predictor? It would retain loop bias through one not-taken outcome and often reduce next-loop startup misses, but requires additional state and policy. The current design is smaller and easier to explain. No branch-accuracy percentage is guaranteed; measure branch executions and mispredictions on a defined workload.

10. Feature 8: Performance and floating-control CSRs

Address	Value
0x001	Accrued floating exception flags
0x002	Stored rounding-mode field frm
0x003	fcsr = frm in bits 7:5 plus flags in bits 4:0
0xC00 / 0xC80	Active core cycle count low/high
0xC01 / 0xC81	Same cycle count, used as a time mirror; not an independent timer
0xC02 / 0xC82	Retired valid instructions low/high
0xC03 / 0xC83	D-cache miss-event count low/high
0xC04 / 0xC84	I-cache miss-event count low/high
0xC05 / 0xC85	Aggregate pipeline stall cycles low/high
0xC06 / 0xC86	Accepted branch mispredictions low/high
0xC07 / 0xC87	Accepted branch/jump executions low/high
0xC08	Latest valid non-x0 register-write data snapshot
0xC09	D-cache hit count, 32-bit
0xC0A	I-cache hit count, 32-bit

These are project event assignments, not a complete privileged CSR implementation. The 0xCxx performance addresses are read views; calling 0xC03 "mhpmcounter3" is imprecise nomenclature. Arbitrary performance-counter writes are not implemented. Unknown CSR reads return zero; illegal access trapping is not complete.

cycle counts edges of core_clk. It stops when UART backpressure or slow-mode gating stops that clock. instret counts valid WB instructions, including branches/stores that do not write a register. A bubble is not retired. A write to x0 is not a register commitment, but the instruction may still retire.

stall increments once when the aggregate stall signal is asserted, even if multiple causes overlap. Do not add all individual busy durations and expect equality with this union. Miss events are counted at initiation, not on every refill cycle; hit counts use accepted requests to avoid counting a held hit repeatedly. Canceled speculative I-fetches can affect miss statistics. The cache-controller counters and CSR counters are separate implementations, not automatically identical under every cancellation scenario.

Why passive counters? They allow measurement without estimating event totals in firmware. Reading and printing them still adds executed instructions and memory accesses. The current CSV reads separate counters in separate instructions, so it is not an atomic multi-counter snapshot. For a general 64-bit read on RV32, a high-low-high retry is useful; the short demo sends low halves and the host uses modulo-2^32 deltas.

11. Feature 9: Bit manipulation

alu_zbb.v shares the EX result path with base integer ALU operations. It provides ANDN, ORN, XNOR; CLZ, CTZ, CPOP; MIN, MINU, MAX, MAXU; SEXT.B, SEXT.H, ZEXT.H; ROL, ROR, RORI; ORC.B and REV8. Each is combinational and adds no dedicated multi-cycle busy state. This audit adds 80 independently generated operand cases for each of the 18 operations, but instruction-encoding legality and exhaustive operand coverage remain incomplete.

Operation	Construction	Teaching example
REV8	Reorders four byte slices	0x0000803F -> 0x3F800000
CPOP	Adds each of 32 bits into a six-bit count	All ones -> 32
CLZ / CTZ	Priority search from MSB / LSB	Zero -> 32
Signed MIN/MAX	Sign-extended subtraction/comparison	max(-1,5) -> 5
Unsigned MINU/MAXU	Zero-extended comparison	maxu(0xFFFFFFFF,5) -> 0xFFFFFFFF
ROL/ROR	Shift duplicated 64-bit operand, choose half	Bits leaving one end re-enter the other
ORC.B	OR-reduce each byte, replicate result	Nonzero byte becomes 0xFF

REV8 changes byte order, not floating magnitude by arithmetic. CPOP returns a count; parity would be count modulo two. The demo's signed MAX is applied to the raw 32-bit pattern, not a general floating-point comparison. It clamps sign-bit-set patterns to zero for this constrained input scheme, but is not a replacement for IEEE floating min/max.

Dedicated bit operations reduce a multi-instruction software sequence to one EX operation. The trade-off is more ALU logic and result selection. Source-level for loops in combinational Verilog describe replicated logic, not a software loop taking 32 clock cycles.

12. Board support: clock, reset, I/O and UART

12.1 Clocking and reset

MMCME2_BASE uses a 100 MHz input with multiplier ten, producing a 1 GHz VCO. At 40 MHz, CLKOUT0 divide is 25. A BUFG distributes the ungated system clock; a BUFGCE derives the core clock. Legacy internal names such as clock50 and pll_clock50 remain even when the actual configured frequency is 40 MHz. The display input named clk_100mhz is also connected to the configurable system clock, not the raw 100 MHz input.

CPU_RESETN and MMCM LOCKED feed asynchronous reset assertion. A two-bit pipeline synchronizes release on the ungated clock. Switches use two flip-flop stages marked ASYNC_REG. This reduces metastability risk but is not a mechanical debouncer or an atomic multi-bit switch transaction. Set mode/cache switches before resetting for reproducible measurements.

The core and backing RAM use the same gated clock, so trace pauses do not let memory responses advance independently of cache state. UART and the display continue on the ungated clock. Slow mode permits at most ten core ticks per second, not ten instructions per second. A 15-cycle FPU operation can alone occupy about 1.5 seconds in slow mode, before other waits.

12.2 MMIO and switches

MMIO address	Firmware purpose
0x80000000	Read synchronized SW[15:0]
0x80000004	Write low byte into software UART line buffer; read busy in bit 31
0x80000008	Write software seven-segment override value
0x8000000C	Write completion bit 0, which controls LED0 and core stopping

MMIO is selected before D-cache: peripheral accesses must not be satisfied from cached data or trigger line refills. Address bits [3:2] select these four registers. The firmware uses aligned word accesses.

Switches	Meaning
SW15:14	00 flight, 01 profiler, 10 cache/branch stress, 11 diagnostics
SW13	Enable both L1 caches; zero bypasses them
SW12	Verbose every-active-cycle trace; zero selects hybrid sampling
SW11	Ten-core-ticks-per-second throttle
SW10	Show software override instead of IF PC
SW9:8	Reserved in the demo
SW7:0	Flight alpha seed or diagnostic vector selector

12.3 LEDs and seven-segment display

LED	Meaning
15	Aggregate pipeline stall
14	Load-use hazard indication
13 / 12	D-cache / I-cache miss-active indication
11	Branch prediction recovery flush
10	FPU busy
9	Divider busy or accepted integer multiply activity
8	Valid non-x0 register write, including floating registers
7	Floating NV event
6	Floating DZ or integer divide-by-zero observation
5 / 4 / 3	Floating OF / UF / NX events
2	Zbb activity indication with exact supported opcode/function qualification
1	UART transport activity
0	Firmware completion flag

LED15:2 are stretched for approximately 20 ms after an asserted input. The snapshot's LED field contains these physical display states; EVENT contains raw event bits. Trace quieting starts around the completion sentinel, before the final completion store; the host LED0 mirror may therefore never receive the final physical LED0=1 snapshot. Observe the board LED for that final indication. Consequently a lit LED is not a count of cycles and can remain on after the triggering event. With a paused core an event input can remain high, stretching the apparent indication further.

The seven-segment display multiplexes eight active-low anodes and cathodes. Digit selection uses counter bits 16:14: at 40 MHz, each digit slot is 16,384 ticks and a full scan is 131,072 ticks, about 305 Hz. SW10 selects the software-written filtered/result word, not a direct live WB bus, despite older comments. Otherwise the display shows IF PC. DP on digit 0 follows retirement and DP on digit 7 follows stall; decimal-point events are not independently stretched.

12.4 Serial transport and backpressure

uart_tx.v shifts a start bit, eight data bits LSB first, then a stop bit. Its divisor is rounded: (CLOCK_HZ + BAUD/2)/BAUD. At 40 MHz, divisor 347 gives approximately 115274 baud. Ten serial bits are transmitted per byte.

uart_trace_transport.v stores up to 128 software bytes until newline/full buffer, and has one pending hardware snapshot. It sends complete lines without mixing software characters into a trace line. core_hold stops further processor ticks while lines are serialized. Trace snapshots have priority over a pending software line.

Hybrid mode captures FP commits and only the first 64 key stall/flush samples. Verbose captures every active core tick until completion quieting; consecutive CC values show active cycles, while hybrid gaps are expected. Each 271-byte trace takes about 23.5 ms at 115200 baud, limiting verbose wall-time progress to about 42 captured cycles/s. Verbose runs can therefore take hours.

13. Firmware workload, modes and completion

13.1 Startup checks

architectural_selftest executes 57 CHECK records covering register/forwarding dependencies, integer multiply/divide cases, Zbb edges, selected floating results and flags, cache spatial/conflict behavior, a loop exit, and active counters. The final SELFTEST_FAILURES record reports the firmware's accumulated failures. A CHECK has actual and expected values; the expected side is a legitimate fixed test-vector oracle, not a measured result.

Passing these checks demonstrates those scenarios. It does not exhaust every instruction encoding, operand pair, cache conflict timing, clock/reset event or exception combination. The firmware continues to the workload even if a self-test fails; the host must treat failed checks as failures.

13.2 Flight data and EMA

The raw sequence is generated in firmware, not read from an IMU or network interface. For packet k, the initial bit pattern is 0x3F800000 + (k mod 16)*0x00100000, with an additional 0x00600000 when k mod 7 is zero. The program byte-reverses it to create a network-order-like buffer, then reverses again when processing. The first packet is 0x3FE00000, or 1.75, after reversal. The comment suggesting negative glitches does not match this positive generated sequence.

alpha = binary32(seed/255). beta = binary32(1-alpha). The first output is the first input. Later outputs are binary32(binary32(alpha*x) + binary32(beta*previous_y)). These are separate rounded multiplications and addition, not a fused operation.

For alpha=0 the filter retains its initialized value; for alpha=1 it follows the input. Smaller alpha usually smooths more but responds more slowly. A simple explanatory example with alpha=0.25, previous output 2 and new input 6 gives output 3. The actual switch value 50 means alpha about 0.19607843, not exactly 0.2.

Flight mode also executes DIVU(iter+100,7), producing quotients 14 through 28. The CSV integer-result field is not a measured Zbb latency despite a stale inline comment. The latest WB field is a CSR snapshot read later in the program, not necessarily the filtered value or the WB event belonging to the same packet's FPU operation.

13.3 Other modes

Profiler mode runs six isolated windows only on packet zero. It measures a back-to-back cycle-read baseline and subtracts it. The load window contains a dependency and correction terms; the D-miss penalty is miss-window minus hit-window. Alignment directives help place the short windows within instruction lines. The checked cache-enabled configuration reports REV8=1, D-hit service=1, D-miss penalty=9, FMUL=4, FDIV=15 and DIV=32. These numbers are measurements under defined conditions, not universal instruction costs including arbitrary cache misses. With caches disabled or a live mode switch after packet zero, the profiler assumptions need not hold.

Stress mode alternates accesses at indices 0 and 512 of an aligned 1024-word array. These locations differ by 2048 bytes and collide in the D-cache. A sixteen-iteration loop adds repeated stores, loads and branches; the telemetry integer result is XORed with read values. This is the implemented stress workload, not a general cache benchmark.

Diagnostics select eleven predefined cases. Seeds 1-5 generate infinities/NaNs, 6 signed negative zero, 7 an inexact multiply rounding example, 8 integer divide-zero, 9 signed overflow, 10 signed/unsigned MAX, 11 packed CLZ/CTZ results and CPOP. The seed-7 multiply is not the strongest proof of an exact halfway tie; the startup TIE_EVEN/TIE_ODD additions are clearer. Other seeds return the switch word as raw bits and perform integer divide-zero; they do not implement a general raw-seed float-conversion demonstration.

Switches are read each packet. The host now updates its independent EMA state across diagnostic/flight transitions, matching firmware state semantics. Arbitrary physical switch transitions have not been comprehensively tested. For reproducible viva runs, select a configuration, clear the dashboard and reset the FPGA.

13.4 Bounded run

After 100 packets, firmware sends DEMO_COMPLETE, waits for software UART readiness, sets status bit 0 and falls into a fallback NOP loop. The board's completion flag then disables the processor clock. Thus physical completion depends on the board wrapper; the C infinite loop alone is not a terminating operating-system process. UART/display remain clocked. Pressing board reset clears completion and restarts execution.

The sentinel arrives before the completion MMIO store. Seeing the sentinel on the laptop does not independently prove LED0 asserted or the core clock stopped; observe LED0 or inspect the corresponding board-level signals.

13.5 Exact UART record formats

All eleven CSV payload fields are eight hexadecimal digits; the word CSV is a prefix, not a numeric field. A line has the form CSV,iteration,switches,raw,result,cpop,integer_result,cycle,instret,dmiss,stalls,last_wb. The newline terminates the record. There is no packet checksum.

Position after CSV	Meaning and origin
1	Iteration, firmware integer 0-99
2	Synchronized switches sampled by firmware for this packet
3	Processed input bits; diagnostic output bits in diagnostic mode
4	EMA result bits, or diagnostic passthrough result
5	Hardware CPOP result
6	Integer divide/diagnostic/stress result
7	Low 32 bits of hardware cycle CSR
8	Low 32 bits of hardware instret CSR
9	Low 32 bits of D-cache miss CSR
10	Low 32 bits of aggregate stall CSR
11	Last non-x0 WB data read from custom CSR 0xC08

CHECK,name,actual,expected records use two eight-digit hexadecimal words. PROFILE,name,value records carry the measured/corrected window result as hexadecimal. DEMO_COMPLETE is a literal end-of-run sentinel. Hardware trace records start with [CC=...] and include WB PC, instruction, destination, data, WE/VALID, a raw EVENT mask, decimal DREFILL progress, switches, stretched LEDs, IFPC and I/D hit/miss counts. CC is decimal in the hardware trace; most address/data metadata is hexadecimal. Converting this textual representation is a host parsing task, not a new hardware computation.

14. Dashboard legitimacy: field-by-field audit

Display item	Actual source/calculation	What it does and does not prove
Cyan input plot	CSV raw bits decoded as binary32	FPGA-reported generated input, already byte-swapped/clamped; no physical sensor
Green filtered plot	CSV filtered bits decoded as binary32	FPGA-reported FPU output in LIVE mode
Host Expected	Python reference described below	Independent selected result computation; not a CPU emulator
CPOP result	FPGA CSV field versus host count of one bits	Checks this result against transmitted input in flight
Integer result	CSV DIV/diagnostic/stress result	Host independently checks diagnostic, normal DIVU and stress XOR results
LIVE IPC	Delta instret / delta cycle between CSV samples	Active-cycle interval throughput, includes firmware overhead
D-cache hit percentage	100*D_HIT/(D_HIT+D_MISS) from latest trace metadata	Cumulative cache-controller event ratio, including stores; no fixed 95%
I-cache hit percentage	100*I_HIT/(I_HIT+I_MISS) from latest metadata	Speculative fetch behavior included; no fixed 99%
Pipeline stalls	CSV hardware aggregate stall count	Cumulative active-core stalls, not UART clock-off duration
IF PC label	Latest trace IFPC	Can be older than the current packet; not an inferred PC
Trace PC/INST/WB	Captured WB-stage state plus valid/write bits	INST disassembly is host formatting of real captured bits
SW / LED mirrors	Latest trace snapshots	SW synchronized; LEDs pulse-stretched; not continuously sampled
CHECK PASS	Actual and declared expected words compared with host-owned dictionary	Requires known unique checks; cache-dependent values also checked against packet-zero switches
PROFILE lines	Firmware CSR measurements printed to terminal	Offline check_profiles.py validates known windows; GUI does not automatically validate them
Completion color	All 100 sequential packets, all 57 unique expected checks and zero failures	Synthetic sources cannot produce verified green completion
Power / voltage / temperature	Not implemented	No live measurement, gauge or valid power estimate exists in dashboard.py
MHz / MIPS / wall runtime	No live measurement widget implemented	Clock frequency comes from build configuration, not a sensor

Fixed gauge maxima (IPC 1.2, hit rate 100%, stall arc 5000), colors, labels and initial placeholder values are UI settings. The stall arc saturates at 5000, but its numeric value continues to show the count. A fixed maximum is not a fabricated measurement. Initial readings and Reset/Clear now show N/A and clear proof/mirror values until data arrives. Host Clear does not reset the FPGA.

Non-finite values are plotted as gaps, because Matplotlib cannot meaningfully plot NaN/infinity on an ordinary finite axis. The proof card still shows hexadecimal bits. Diagnostic integer bit patterns may

decode as unusual floats; interpret the diagnostic field by its seed, not by the graph's float label alone.

The Synthetic flight and Synthetic fault buttons deliberately generate Python data, artificial cycle/instruction/stall totals and a completion sentinel. Their source is marked SYNTHETIC - NOT HARDWARE VERIFICATION. They are for UI demonstration only. Their generator still uses arbitrary +11 stall and +23/+13 cycle increments; those are not the nine-stall cache model and must never be used as timing evidence. The internal description now correctly identifies synthetic behavior, and synthetic completion remains amber rather than verified green.

15. How the laptop computes the expected value

HostGoldenModel runs on the laptop CPU through the Python interpreter. Python floats normally use binary64. f32 packs to a four-byte IEEE float and unpacks again, forcing a binary32 rounding boundary. It does not send computation back to the FPGA or merely copy the filtered-output field.

For a normal packet:

1. Decode the raw hexadecimal bits into a binary32 value represented in Python.

2. Compute alpha from the seed and round to binary32; compute and round 1-alpha.

3. On packet zero, initialize host EMA state to raw input.

4. Otherwise multiply alpha by raw input and round; multiply beta by previous host EMA and round; add the two rounded terms and round.

5. Convert the expected value back to 32-bit hexadecimal, canonicalizing NaN to 0x7FC00000.

6. Compare exact result bits with the FPGA result and compare CPOP with the host one-count.

The reference state is updated from the host's own previous result, so a bad FPGA filtered result is not fed back as the next expected value. The live host and offline checker independently reconstruct the deterministic raw-input sequence. The live parser also validates normal integer division and stress-mode XOR results. Diagnostic expected results update the reference EMA state before a subsequent flight packet.

For diagnostics the host uses a fixed expected-result table keyed by the seed. This is appropriate for fixed inputs and specified corner cases: Python division by zero would otherwise raise an exception. CHECK records are independently validated against a host-owned dictionary of 57 names and allowed expected values. Unknown, duplicate or spoofed checks fail. Cache-dependent startup checks are reconciled against packet-zero switches, assuming the cache switch stays stable during startup.

The binary64-plus-f32 reference is useful for the finite constrained demo, but it is not a formal all-input floating-point oracle. Overflow is mapped to signed infinity; NaN payload/signaling distinctions, rounding-mode behavior, exception flags and double-rounding corner cases require more care for a general test system. The separate FPU differential tests use NumPy float32 results on 2,784 operand/operation cases. They compare result bits, while directed tests cover selected flags; neither establishes exhaustive compliance.

The dashboard's disassemble function extracts opcode, rd, rs1, rs2, funct fields and immediates from the captured instruction word. It is a convenience decoder for supported instructions, not a full ISA decoder. A trace line's PC and INST describe WB, while IFPC describes fetch and event flags describe concurrent pipeline activity. Do not assume an FPU busy flag belongs to the instruction shown in WB on the same line.

16. Timing, speed and power: defensible interpretation

At 40 MHz, an active clock cycle is 25 ns. If a measured window uses C core cycles, its active-clock duration is C/40,000,000 seconds. With I retired instructions, IPC=I/C and CPI=C/I. A hypothetical sustained active-core instruction rate would be IPC*40 million instructions/s. This is not the demonstrated UART-limited wall-time throughput and does not make the processor comparable to a laptop CPU without a common workload.

Wall time includes trace serialization, software serial output, slow-mode gating and host handling. cycle does not count intervals when the core clock is disabled. For bytes B at nominal 115200 baud, wire time alone is about B*10/115200 seconds. A pure algorithm speed benchmark should isolate the computation window and define how instrumentation affects the result.

Implementation	Evidence available	Interpretation
Earlier 50 MHz	Final WNS -3.722 ns; TNS -836.123 ns; 446 failing endpoints	Timing failure
Optimized 50 MHz	Final WNS -0.941 ns; TNS -39.382 ns; 96 failing endpoints	Improved but still failing
Latest 40 MHz	Router estimate WNS +0.361 ns, WHS +0.019 ns; final negative-path searches empty	Supplied run passed those setup/hold checks

The 40 MHz numbers are router estimates in the console, not a newly inspected final timing-summary table. The script's post-route searches found no negative max/min paths. Physical behavior, complete I/O timing review and final bitstream DRC remain separate checks. Prior timing summaries also noted ports without input/output delay constraints; an internal timing pass is not a universal external-interface timing guarantee.

A multicycle FPU can still fail timing: every individual register-to-register step must settle within one clock period, allowing for clock uncertainty and setup time. Four-cycle FMUL is not permission for its normalization logic to take four clock periods unless that path is architecturally and correctly constrained as such. The delivered flow did not hide these failures with blanket false paths or multicycle exceptions.

There is no power sensor telemetry. Vivado's report_power output, if used later, is a model-based estimate dependent on activity assumptions, not measured watts. The provided console shows vectorless activity propagation during power estimation, but does not supply a power value for this guide. Lower frequency can reduce some switching activity, yet no quantitative power-saving claim is supported here.

17. Audit results, reproduced gaps and limits

17.1 What was checked now

The active architecture, board, firmware and dashboard were reviewed against the master specification and the later nine-stall correction. The supplied 40 MHz log was inspected. Before the arithmetic correction, the existing regression was re-run successfully: 19 core integration checks, 12 directed FPU cases, 2,784 differential FPU results, 20,000 packer equivalence cases, 16 integer edge checks, cache timing/bypass/toggle, UART, trace transport, display and eight headless dashboard tests. The board wrapper elaborated successfully.

The firmware was rebuilt in a separate directory. The resulting .bin and .mem match the delivered files byte-for-byte. The delivered memory-image SHA-256 is 7fa6f80d97e185e9c17ec971d6821b63ef9e909280657fa1caeae578d4654fb7. This checks source/image alignment, not correctness of every instruction.

The 15 stored final-mode transcripts were rechecked: four mode configurations plus eleven diagnostic seeds, each with 100 packets, 57 CHECK records and no detected mismatch. These are existing simulation transcripts, not 15 newly executed FPGA runs or a newly repeated full mode simulation. Before the correction, stored 40/50 MHz board simulations also reported 100 packets and 57 checks with zero host failures. Their UART divisor is accelerated for simulation, so they verify serialization/control/data flow rather than real elapsed serial time. GUI visual desktop testing and physical FPGA testing have not been completed.

17.2 Arithmetic defect found and corrected

An additional independent integer-vector audit exercised all 18 listed Zbb operations and ADD/SUB/SRA/SRAI/SRL/SLT, with 80 deterministic cases per operation. Of 1,920 cases, 85 failed in the original ALU; the failures were arithmetic right shifts of negative operands. For example, SRA of 0xF10D27C8 by 30 returned 0x00000003 instead of 0xFFFFFFFF.

Cause: the arithmetic shift expression was inside a conditional expression with an unsigned logical-shift alternative. Verilog expression signedness caused the right shift to zero-fill. Correction: compute the signed arithmetic shift in its own wire, then select the already computed 32-bit result. This preserves sign filling without changing the pipeline architecture or instruction latency.

The same 1,920 cases then passed with zero failures. tests/tb_alu_audit.v retains the vectors and is included by tools/run_tests.py. This test is independent of the ALU implementation: the expected words were generated using Python integer arithmetic. It is additional operand coverage, not exhaustive 32-bit formal equivalence. The existing firmware image is unchanged and rebuilt identically. Fresh post-fix regression and board evidence are summarized in the revision record below.

The audited revision additionally fixes dashboard validation, Zbb observer decoding and trace stall labeling, as detailed below. A new Vivado run at 40 MHz is required for the corrected hardware; old positive slack does not certify the new netlist.

17.3 Reproduced host-validation defects

ID	Before audit fix	Implemented correction
H1	Feed completion with received=100, failures=0, check_count=0: status becomes green	Now requires all 57 unique check names, 100 sequential packets and zero failures; synthetic runs remain unverified
H2	Replace normal packet DIV field 0000000E with DEADBEEF: parser failures remains zero	Now validates DIVU(iter+100,7) and models stress-mode XOR independently
H3	Send CHECK,MUL,00000001,00000001: parser reports match	Now checks both words against a host-owned expected dictionary
H4	Flight packet, diagnostic negative-zero packet, then flight: host predicts 3F800000 but firmware's state implies 3E48C8C9 in the tested example	Now updates reference EMA state with independently expected diagnostic output

All four findings are repaired and covered by the expanded 20-test headless dashboard suite. The 15 stored final-mode transcripts also pass the stricter parser. A green GUI verifies these scoped protocol/result checks, not all possible processor behavior or physical timing.

17.4 Further source-level limitations

- **FPU compliance:** missing F instructions, incomplete non-RNE support and missing FCVT inexact flags. Small seed conversions 0-255 are exact, so the last issue is not exposed by flight alpha conversion.
- **Illegal instructions and traps:** illegal_inst_o is decoded but is not a complete exception/trap subsystem. SYSTEM decoding and unsupported encodings are permissive. No OS, privilege modes, interrupts or MMU are demonstrated.
- **Zbb LED accuracy, repaired:** exact supported opcode/function qualification excludes unrelated ADDI encodings; a new 24-case observer test covers positives, false classifications, held and invalid instructions.
- **Miss label accuracy, repaired:** a D-cache miss is labeled a wait only when aggregate stall is also asserted. Eight trace cases include a non-stalling store miss. Overlapping events still use a priority label; inspect raw EVENT bits to distinguish concurrent causes.
- **Raw-input verification, repaired:** live host now independently validates the known generated input sequence as well as the filter result.
- **Reset UI, repaired:** host Clear resets gauge/proof/mirror readouts to unavailable. It does not reset the FPGA; press board reset for a fresh run.
- **Record loss/reconnection:** no explicit run identifier, checksum or resend protocol. A missing/out-of-order CSV raises an error, but reconnecting mid-run does not reconstruct reference history. Serial exceptions are largely retried without detailed diagnostics.
- **Trace completeness:** hybrid misses most later per-cycle events; terminal retention is about 10,000 lines; there is no built-in full-session trace archive or automatic per-miss timing checker. I-cache refill progress is not explicitly exported.
- **Memory semantics:** no arbitrary-latency memory handshake, misaligned access splitting or instruction-cache coherence for modified code. Out-of-range accesses do not become a full architectural fault.
- **Parameterization:** several address slices remain fixed to the shipped cache/BTB geometry. RESET_PC is passed through wrappers but fetch_stage resets PC to literal zero. Do not assume editing one parameter safely creates another configuration.

- **Profiler assumptions:** meaningful known windows use the initial cache-enabled profiler run. Switching into profiler after packet zero does not execute its special window block.

17.5 Specification alignment

Original expectation	Current implementation / status
Nine feature groups exercised	Demonstrated by scoped tests and firmware; not exhaustive certification
Nine stalls / ten occupancy for both caches	Direct no-contention tests pass; extra arbitration waits possible
Firmware prints each refill cycle	Replaced with real RTL trace; DREFILL only, no fabricated software timing loop
Full RV32F / all rounding semantics	Not satisfied; demo subset/RNE scope
Backward static predictor	Different: learned taken-target BTB
95%+ cache hits and high IPC	Not guaranteed; ratios come from actual workload events
Every cycle visible	Verbose active cycles only; hybrid is sampled; clock pauses during transport
Slow mode about ten instructions/s	Actually at most ten core ticks/s
SW10 live WB display	Actually software MMIO override
"9-field" CSV	Eleven numeric fields, matching the enumerated data requirements
Power/speed gauges from mockups	No power or live-frequency measurement implemented
Production-ready/error-free	Not established; concrete remaining defects listed above

17.6 What would strengthen signoff

H1-H4, raw-input validation, observer qualification and trace reason fixes are completed. Next extend instruction/flag tests for conversion, non-RNE modes and unsupported encodings before expanding ISA claims. Test reset/restart, live switches and actual USB-UART on the board. Any RTL fix requires a fresh routed timing run.

18. Viva questions with answer guidance

Architecture and pipeline

1. What is implemented in hardware versus software? The pipeline, arithmetic, caches, predictor, counters, RAM interface and UART are RTL hardware. C is the workload executed by that hardware. Python visualizes and checks selected results.

2. Why use five stages? It exposes instruction overlap with manageable register/control boundaries. It is an educational area/complexity trade-off, not proof of an optimal stage count.

3. Is every instruction finished in five cycles? No. Cache misses and multicycle EX operations hold stages. Even a simple instruction's latency depends on surrounding hazards.

4. Why forward from EX/MEM before MEM/WB? If both destinations match, EX/MEM is the newer older producer and must override the older WB value.

5. Why cannot forwarding eliminate a load-use bubble? The load's EX output is an address. The loaded value becomes available later; the dependent instruction must wait before using it.

6. What is the difference between stall and flush? Stall preserves work; flush invalidates work. A flush must disable side effects, not merely change the PC display.

7. Why six-bit register IDs? They distinguish xN from fN. Only identifier zero is x0; f0 is identifier 32 and must remain writable.

8. Why snoop WB into a held ID/EX register? A value can retire while the consumer is stalled. Saving it avoids losing a transient bypass after WB moves on.

Arithmetic

9. Why is MULH(-1,-1) zero while MULHU gives FFFFFFFE? The high halves reflect different signed interpretations of identical input bits. The full signed product is one; the unsigned product is 0xFFFFFFFE00000001.

10. Why use DSP multiplication? It maps arithmetic to dedicated FPGA resources. The board selects it; Booth is an alternative source implementation, not an active switchable demo feature.

11. How does the divider work? Binary long division reuses shift/compare/subtract hardware across quotient bits, then adjusts signs.

12. Does divide by zero hang or trap? Integer division returns defined quotient/remainder values by an early path. Floating division produces specified infinity/NaN behavior and flags for the supported cases.

13. How can a four-cycle FPU still fail 50 MHz timing? One internal combinational step can exceed 20 ns. Total instruction latency and each register-to-register timing budget are different constraints.

14. Is the FPU pipelined? It has internal staging registers but accepts one operation at a time. It is not a throughput pipeline accepting independent operations each clock.

15. What do guard, round and sticky mean? They summarize discarded precision so rounding can choose the nearest representable value and resolve halfway ties using the retained LSB.

16. Is every tiny number an underflow exception? No. Exact subnormal results can be valid without UF; tiny inexact results are the important exception case here.

17. Are all F-extension instructions implemented? No. Name the implemented operations and openly state missing fused, square-root, comparison/classification and conversion features.

18. Does writable frm prove rounding-mode compliance? No. The state field exists, but multiply/divide use an RNE-only final packer and the validation scope is RNE.

Caches and prediction

19. Why is the cache 2 KiB? 128 lines x 4 words x 4 bytes. Metadata is additional; FPGA resource use is measured separately.

20. Why do addresses 2 KiB apart conflict? Their index[10:4] matches while the tag differs; a direct-mapped cache has only one location for both lines.

21. Why four words per line? To exploit spatial locality with a modest refill size. Subsequent accesses within the line can hit.

22. Is the RAM physically a ten-cycle memory? No. The refill FSM models slow service on top of synchronous on-chip RAM for the cache experiment.

23. Is the miss penalty nine or ten? Nine extra stall cycles; ten cycles of total occupancy, including service. State which number you mean.

24. Why can more than nine stalls occur? Arbitration can delay requests, other hazards can overlap, and UART clock pauses lengthen wall time. Nine is the no-contention refill baseline.

25. Does every D-cache miss cost nine stalls? No. Cached no-write-allocate store misses can write through without a refill wait when granted.

26. Why write-through rather than write-back? It avoids dirty-line eviction machinery and keeps backing data current. It generates more memory write traffic and does not provide a general store queue here.

27. Does D-cache always preempt I-cache? No. D wins idle contention; an already owned I burst completes before handoff.

28. Is the predictor backward-taken by construction? No. It predicts taken on a valid matching learned target. It neither tests branch direction nor uses two-bit counters.

29. What happens at loop exit? EX detects disagreement, redirects to fall-through and invalidates the two younger pipeline slots. The BTB entry is invalidated by the not-taken update.

Measurement and demonstration

30. Are the plots actual sensor readings? No. They are real FPGA computations on deterministic firmware-generated inputs in LIVE mode.

31. Are host expected values hardcoded? Normal EMA is recomputed by Python with binary32 rounding boundaries. Diagnostic vectors use a fixed independent table. CHECK words are now checked against a host-owned expected dictionary, including startup cache-mode consistency.

32. What exactly is IPC here? Delta retired instructions divided by delta active core cycles between samples. It includes firmware work, and excludes time while the core clock is off.

33. Are cache hit percentages estimated from misses alone? No. Trace metadata includes hit and miss counts. The GUI uses hits/(hits+misses), with N/A when no denominator is available.

34. Can the dashboard measure watts? No. No physical power measurement exists. Vivado estimates and external power measurements are separate kinds of evidence.

35. Why pause the processor for UART? Serial bandwidth is much lower than the potential trace generation rate. Backpressure prevents the small snapshot buffer from being overwritten.

36. Does the PC on a trace line identify the currently stalled instruction? Not necessarily. The line's PC/INST are WB observations; event bits describe concurrent pipeline activity, and IFPC is separately reported.

37. Why does an LED stay on for many cycles? Pulses are stretched for visibility. LEDs are qualitative event indicators, not cycle counters.

38. What proves completion? The expected packet/check records plus sentinel show protocol completion; LED0 and a stopped core clock confirm the wrapper's terminal state. The GUI now enforces all 57 unique checks and 100 sequential packets before verified completion.

39. Can you guarantee no bugs? No. State the tests passed, the timing evidence, the known limitations and the remaining board checks. That is stronger engineering evidence than an unsupported guarantee.

40. What would you improve next? Complete trace semantics and broader ISA/flag coverage, robust trap handling, more rigorous cache/bus interaction tests and possibly predictor/cache alternatives measured on common workloads.

19. Suggested live demonstration and team rehearsal

Start with 0x2032: SW13, SW5, SW4 and SW1 on, others off. This selects caches enabled, flight mode, alpha seed 50 and hybrid trace. Start the dashboard on the board's COM port at 115200, clear host state, then reset the board. Inspect the checks, input/output plots, exact-match records and completion. Do not use Synthetic buttons as hardware evidence.

For cache comparison, reset and run 0x0032. Compare a defined active-cycle/instruction window and actual event counts; do not compare only stopwatch times dominated by UART. For profiler use a fresh 0x6032 run with caches enabled. For conflict stress use 0xA032. For diagnostics choose 0xE001 through 0xE00B, clearing and resetting between configurations. Avoid a full verbose run during a short viva; capture a limited prefix or explain its bandwidth cost. Slow mode is also unsuitable for completing the full workload quickly.

Suggested team ownership: one person explains pipeline/register hazards; another arithmetic/FPU; another caches/BTB/counters; everyone rehearses the board-to-host data path and limitations. Each person should locate their module, identify its inputs/outputs, explain one normal case and one edge case, and name the corresponding test.

A good presentation sentence is: "At 40 MHz our routed build passed the script's setup/hold checks, and our tested workload completes in simulation. These are actual hardware counters and output bits when using the live serial source. We are still validating physical operation and do not claim full ISA compliance or measured power."

20. Source map and evidence references

File paths below are relative to the delivered project root. Code behavior takes precedence over comments. The original master prompt includes illustrative firmware/dashboard code; it is a requirements/history document, not the Python or C file that actually executes.

External references used only for ISA semantics: RISC-V official F extension, https://docs.riscv.org/reference/isa/unpriv/f-st-ext.html ; RISC-V official M extension, https://docs.riscv.org/reference/isa/v20240411/unpriv/m-st-ext.html . These establish the distinction between implemented demo scope and full architectural requirements, particularly conversion flags and integer division corner cases. Most specifications and all audit findings in this guide come directly from this project's source/tests.

The latest user-supplied 40 MHz console is Pasted text(20261104-063504).txt. The earlier detailed optimized 50 MHz reports are Pasted text(20261104-032318).txt and Pasted text (2).txt. Stored project reviews predate the successful 40 MHz run, so their statements that all Vivado implementation is unverified are stale. The new physical-FPGA verification requirement remains valid.

20.1 Every active implementation file

The following catalogue covers all 209 files in the baseline ZIP, plus the audit additions described in the revision record. Reports are evidence/artifacts, not logic synthesized into the FPGA. Testbenches and Python tests also stay off the FPGA. Paths are exact; a file being included in the project does not imply all its modules survive synthesis.

Active design, firmware and project files

Exact file	Role and interpretation
MASTER_SPEC.md	Original requested requirements and illustrative code; not executed. Later user correction overrides ten-stall wording.
README.md	Build/run guide and earlier evidence notes; some pre-40-MHz status statements are historical.
avionics_demo.mem	65,536-byte backing-memory initialization represented as 16,384 hex words; consumed by board RTL.
board/Nexys-A7-100T-Master.xdc	Reference board pin assignments; not the selected constraints file in the script.
board/build_vivado.tcl	Vivado project creation, frequency selection, implementation, reports and timing gate; no bitstream generation.
board/nexys_a7.xdc	Active clock and physical pin/I/O-standard constraints used by the build.
board/nexys_a7_io_controller.v	LED pulse stretching and active-low eight-digit display scanning.
board/nexys_a7_top.v	Physical top; MMCM/reset/gating, 64 KiB RAM, MMIO, core and peripheral wiring.
board/uart_trace_transport.v	Atomic software-line buffering, hardware snapshots, trace formatting and core backpressure.
board/uart_tx.v	UART start/data/stop-bit serializer and configurable baud divider.
checkpoints/review_v1.zip	Historical checkpoint; do not build it expecting the latest corrections.
dashboard.py	Host serial/UI/reference/disassembly implementation; LIVE versus synthetic sources.
firmware/Makefile	RISC-V cross-build and image/disassembly generation recipe.
firmware/avionics_demo.bin	Raw loadable byte image; input to word-hex conversion.
firmware/avionics_demo.c	Actual bare-metal checks, four modes, sensor-like inputs, EMA, profiles and MMIO output.
firmware/avionics_demo.dis	Disassembled compiled instructions for checking compiler output.
firmware/avionics_demo.elf	Linked executable with sections/symbols; input to objcopy and objdump.
firmware/link.ld	64 KiB RAM placement, entry point, BSS symbols and stack-space assertion.
firmware/start.S	Reset program entry, gp/sp setup, BSS zeroing, call main.
integration/modules/arith/alu_zbb.v	Base integer ALU and 18 Zbb operations; SRA/SRAI signedness corrected in this audit.
integration/modules/arith/booth_radix4_multiplier.v	Combinational 17-group Booth alternative; board selects the other path.

Exact file	Role and interpretation
integration/modules/arith/divider_unit.v	Iterative quotient/remainder FSM with signed adjustment and early corner cases.
integration/modules/arith/fpu_single_core.v	Single-operation-at-a-time FADD/FSUB/FMUL/FDIV FSM and result/flag logic.
integration/modules/arith/multiplier_dsp.v	33-bit sign/zero extension and DSP-directed product inference.
integration/modules/arith/multiplier_unit.v	Operand signedness, DSP/Booth result selection and low/high product selection.
integration/modules/branch/btb_branch_pred.v	32-entry valid/tag/target array; taken-on-hit query and EX update.
integration/modules/cache/dcache_controller.v	Direct-mapped data cache, write-through, no-write-allocate, delayed bypass/refill FSM.
integration/modules/cache/icache_controller.v	Direct-mapped instruction cache, refill/bypass, fetch cancellation and counts.
integration/modules/cache/mem_bus_arbiter.v	Shared RAM-port owner, idle D-cache priority and active-burst ownership.
integration/modules/core/pipeline_5stage.v	Complete stage wiring, valid/control paths, hazards, cache/MMIO routing, CSRs and observation signals.
integration/modules/core/rv32imf_core.v	Public processor interface wrapper around pipeline_5stage.
integration/modules/csr/csr_perf_counters.v	Cycle/event counters, last-WB snapshot, floating-control CSR storage.
integration/modules/csr/opcode.vh	Shared instruction-field, opcode/function, forwarding and CSR constants.
integration/modules/hazard/hazard_unit.v	RAW comparisons, forwarding priority and stage stall/flush equations.
integration/modules/registers/ex_mem_reg.v	Stores EX result, address/store payload and MEM controls.
integration/modules/registers/id_ex_reg.v	Stores decoded instruction/operands/control/prediction; snoops WB while held.
integration/modules/registers/if_id_reg.v	Stores fetched instruction, PC and prediction with valid/stall/flush.
integration/modules/registers/mem_wb_reg.v	Stores memory/result data and WB metadata; bubble handling.
integration/modules/stages/decode_stage.v	Opcode/immediate decode and two-bank logical register storage with WB bypass.
integration/modules/stages/execute_stage.v	Forwarding muxes, arithmetic selection, branch resolution, conversion/sign injection and CSR source.
integration/modules/stages/fetch_stage.v	PC register, PC+4, BTB lookup and recovery target selection.
integration/modules/stages/memory_stage.v	Address output and store data/byte-strobe formatting.
integration/modules/stages/writeback_stage.v	Load sign/zero extension and final register-data selection.
integration/testbench/integration_prog.S	Small directed integration-test assembly program; separate from avionics firmware.

Exact file	Role and interpretation
integration/testbench/integration_prog.hex	Machine words for the small integration test; not the board demo image.
integration/testbench/memory_model.v	Simulation-only backing-memory model used by the core integration test.
integration/testbench/tb_integrated_core.v	Directed processor integration test, 19 reported checks.
original_source.zip	Untouched original processor archive, for provenance; not the active revision.
requirements.txt	Python GUI and serial dependencies: customtkinter, pyserial, matplotlib.
MANIFEST_SHA256.json	Package file-integrity hashes; regenerate after changing packaged files.

Test and utility files

Exact file	Role and interpretation
tests/check_profiles.py	Validates six known cache-enabled profiler results from a transcript.
tests/fp_vectors.mem	2,784 stored op/operand/reference-result vectors for binary32 arithmetic.
tests/generate_fp_vectors.py	Deterministic edge/random vector generation using NumPy float32.
tests/run_modes.py	Runs firmware simulation for four mode settings and eleven diagnostic seeds; checks records.
tests/tb_board.v	Board/UART smoke test through first complete CSV, with accelerated serial clock.
tests/tb_board_full.v	Full board serialization/control test: 100 packets, 57 checks, trace presence and LED0; accelerated UART.
tests/tb_board_verbose.v	Verbose trace prefix/consecutive-cycle capture test; not a whole-run verbose qualification.
tests/tb_cache_bypass_store.v	Delayed bypass-store payload/timing/write behavior test.
tests/tb_cache_latency.v	I/D cold and bypass latency for all word offsets; nine stalls and 36-stall total.
tests/tb_cache_toggle.v	Bypass-store stale-line invalidation and sampled refill-mode behavior.
tests/tb_display.v	Display/LED behavior test.
tests/tb_firmware.v	Core plus simulated RAM/MMIO; captures firmware output directly, without physical UART serialization.
tests/tb_fpu_edges.v	Twelve directed FPU result/flag cases.
tests/tb_fpu_vectors.v	Reads fp_vectors.mem and checks arithmetic result bits.
tests/tb_integer_edges.v	Sixteen directed divider/multiplier/Zbb edge checks.
tests/tb_pack_equivalence.v	Compares optimized pack_rne with its earlier implementation on 20,000 cases.
tests/tb_trace_transport.v	Directed framing, snapshot, arbitration/backpressure transport tests.
tests/tb_uart.v	UART bit-level transmitter test.
tests/test_dashboard.py	Twenty headless parser/reference tests using widget stubs; not a rendered GUI test.
tests/verify_board_transcript.py	Feeds a stored board transcript into host parser and requires 100 packets, 57 checks, zero failures.
tests/verify_transcripts.py	Checks stored 15-mode outputs through parser and independently verifies generated flight input bits.
tools/bin_to_mem.py	Little-endian binary-to-hex-word converter with 64 KiB size check/padding.

Exact file	Role and interpretation
tools/check_core.ys	Yosys hierarchy/process/structural checks and statistics for processor RTL.
tools/check_trace.ys	Yosys structural checks/statistics for trace transport.
tools/package.py	Builds deliverable ZIP, writes integrity manifest and checks archive.
tools/render_dashboard_preview.py	Pillow recreation/video renderer using transcript data; not actual GUI screen capture.
tools/run_board_tests.py	Compiles/runs verbose-prefix and full-board RTL benches.
tools/run_tests.py	Main directed regression, headless dashboard tests and board/firmware elaboration; now includes ALU audit.
tools/run_timing_validation.py	Runs board functional simulations at 50 and 40 MHz and checks host-parsed outputs; not static timing analysis.

Recorded evidence and historical artifacts

Exact file	Role and interpretation
reports/BUILD_LOG_REVIEW.md	Earlier engineering review; historical scope/status may be superseded by this handbook.
reports/REQUIREMENTS_REVIEW.md	Earlier engineering review; historical scope/status may be superseded by this handbook.
reports/TIMING_OPTIMIZATION.md	Earlier engineering review; historical scope/status may be superseded by this handbook.
reports/baseline.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/board_full.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/board_full_host_check.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/board_full_uart.log	Stored serial-decoded RTL board transcript; testbench UART is accelerated.
reports/board_smoke.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/board_uart.log	Stored serial-decoded RTL board transcript; testbench UART is accelerated.
reports/board_verbose.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/build_review_board.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/build_review_regression.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/cache_bypass_store.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/cache_toggle.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/display.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/final/uart_0032.log	Stored firmware MMIO-output transcript for switch word 0x0032; historical/final according to directory, not a new FPGA run.

Exact file	Role and interpretation
reports/final/uart_0032.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final/uart_2032.log	Stored firmware MMIO-output transcript for switch word 0x2032; historical/final according to directory, not a new FPGA run.
reports/final/uart_2032.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final/uart_6032.log	Stored firmware MMIO-output transcript for switch word 0x6032; historical/final according to directory, not a new FPGA run.
reports/final/uart_6032.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final/uart_a032.log	Stored firmware MMIO-output transcript for switch word 0xA032; historical/final according to directory, not a new FPGA run.
reports/final/uart_a032.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final/uart_e001.log	Stored firmware MMIO-output transcript for switch word 0xE001; historical/final according to directory, not a new FPGA run.
reports/final/uart_e001.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final/uart_e002.log	Stored firmware MMIO-output transcript for switch word 0xE002; historical/final according to directory, not a new FPGA run.
reports/final/uart_e002.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final/uart_e003.log	Stored firmware MMIO-output transcript for switch word 0xE003; historical/final according to directory, not a new FPGA run.
reports/final/uart_e003.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final/uart_e004.log	Stored firmware MMIO-output transcript for switch word 0xE004; historical/final according to directory, not a new FPGA run.
reports/final/uart_e004.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final/uart_e005.log	Stored firmware MMIO-output transcript for switch word 0xE005; historical/final according to directory, not a new FPGA run.

Exact file	Role and interpretation
reports/final/uart_e005.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final/uart_e006.log	Stored firmware MMIO-output transcript for switch word 0xE006; historical/final according to directory, not a new FPGA run.
reports/final/uart_e006.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final/uart_e007.log	Stored firmware MMIO-output transcript for switch word 0xE007; historical/final according to directory, not a new FPGA run.
reports/final/uart_e007.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final/uart_e008.log	Stored firmware MMIO-output transcript for switch word 0xE008; historical/final according to directory, not a new FPGA run.
reports/final/uart_e008.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final/uart_e009.log	Stored firmware MMIO-output transcript for switch word 0xE009; historical/final according to directory, not a new FPGA run.
reports/final/uart_e009.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final/uart_e00a.log	Stored firmware MMIO-output transcript for switch word 0xE00A; historical/final according to directory, not a new FPGA run.
reports/final/uart_e00a.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final/uart_e00b.log	Stored firmware MMIO-output transcript for switch word 0xE00B; historical/final according to directory, not a new FPGA run.
reports/final/uart_e00b.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/final_mode_results.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/final_modes.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/final_modes_v2.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.

Exact file	Role and interpretation
reports/firmware_run.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/firmware_trace.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/firmware_uart.log	Stored serial-decoded RTL board transcript; testbench UART is accelerated.
reports/fpu_edges.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/fpu_vectors.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/integer_edges.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/integration.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/mode_results.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/pack_equivalence.log	Timing-revision functional simulation/equivalence evidence; frequency names do not imply static timing signoff.
reports/profiler_final.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/profiler_v2.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/profiler_v2.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/profiler_v2_run.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/regression.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/regression_final.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/rtl_changes.patch	Source delta versus original processor; updated on packaging.
reports/selftest_flight.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.

Exact file	Role and interpretation
reports/selftest_flight.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/selftest_flight_run.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/tb_board_full.log	Directed testbench output for board_full. Existing files are overwritten when that bench is rerun.
reports/tb_board_verbose.log	Directed testbench output for board_verbose. Existing files are overwritten when that bench is rerun.
reports/tb_cache_bypass_store.log	Directed testbench output for cache_bypass_store. Existing files are overwritten when that bench is rerun.
reports/tb_cache_latency.log	Directed testbench output for cache_latency. Existing files are overwritten when that bench is rerun.
reports/tb_cache_toggle.log	Directed testbench output for cache_toggle. Existing files are overwritten when that bench is rerun.
reports/tb_display.log	Directed testbench output for display. Existing files are overwritten when that bench is rerun.
reports/tb_fpu_edges.log	Directed testbench output for fpu_edges. Existing files are overwritten when that bench is rerun.
reports/tb_fpu_vectors.log	Directed testbench output for fpu_vectors. Existing files are overwritten when that bench is rerun.
reports/tb_integer_edges.log	Directed testbench output for integer_edges. Existing files are overwritten when that bench is rerun.
reports/tb_integrated_core.log	Directed testbench output for integrated_core. Existing files are overwritten when that bench is rerun.
reports/tb_trace_transport.log	Directed testbench output for trace_transport. Existing files are overwritten when that bench is rerun.
reports/tb_uart.log	Stored serial-decoded RTL board transcript; testbench UART is accelerated.
reports/timing_board_40000000.log	Timing-revision functional simulation/equivalence evidence; frequency names do not imply static timing signoff.
reports/timing_board_40000000_uart.log	Stored serial-decoded RTL board transcript; testbench UART is accelerated.
reports/timing_board_50000000.log	Timing-revision functional simulation/equivalence evidence; frequency names do not imply static timing signoff.
reports/timing_board_50000000_uart.log	Stored serial-decoded RTL board transcript; testbench UART is accelerated.
reports/timing_board_results.log	Timing-revision functional simulation/equivalence evidence; frequency names do not imply static timing signoff.
reports/timing_profiler_results.log	Timing-revision functional simulation/equivalence evidence; frequency names do not imply static timing signoff.

Exact file	Role and interpretation
reports/timing_profiler_run.log	Timing-revision functional simulation/equivalence evidence; frequency names do not imply static timing signoff.
reports/timing_revision/uart_6032.log	Stored firmware MMIO-output transcript for switch word 0x6032; historical/final according to directory, not a new FPGA run.
reports/timing_revision/uart_6032.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/timing_revision_tests.log	Timing-revision functional simulation/equivalence evidence; frequency names do not imply static timing signoff.
reports/trace_transport.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/transcript_verification.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/uart.log	Stored development/regression output. Prefer named final/timing-revision evidence; not all earlier runs describe current code.
reports/uart_0032.log	Stored firmware MMIO-output transcript for switch word 0x0032; historical/final according to directory, not a new FPGA run.
reports/uart_0032.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/uart_2032.log	Stored firmware MMIO-output transcript for switch word 0x2032; historical/final according to directory, not a new FPGA run.
reports/uart_2032.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/uart_6032.log	Stored firmware MMIO-output transcript for switch word 0x6032; historical/final according to directory, not a new FPGA run.
reports/uart_6032.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/uart_a032.log	Stored firmware MMIO-output transcript for switch word 0xA032; historical/final according to directory, not a new FPGA run.
reports/uart_a032.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/uart_e001.log	Stored firmware MMIO-output transcript for switch word 0xE001; historical/final according to directory, not a new FPGA run.

Exact file	Role and interpretation
reports/uart_e001.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/uart_e002.log	Stored firmware MMIO-output transcript for switch word 0xE002; historical/final according to directory, not a new FPGA run.
reports/uart_e002.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/uart_e003.log	Stored firmware MMIO-output transcript for switch word 0xE003; historical/final according to directory, not a new FPGA run.
reports/uart_e003.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/uart_e004.log	Stored firmware MMIO-output transcript for switch word 0xE004; historical/final according to directory, not a new FPGA run.
reports/uart_e004.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/uart_e005.log	Stored firmware MMIO-output transcript for switch word 0xE005; historical/final according to directory, not a new FPGA run.
reports/uart_e005.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/uart_e006.log	Stored firmware MMIO-output transcript for switch word 0xE006; historical/final according to directory, not a new FPGA run.
reports/uart_e006.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/uart_e007.log	Stored firmware MMIO-output transcript for switch word 0xE007; historical/final according to directory, not a new FPGA run.
reports/uart_e007.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/uart_e008.log	Stored firmware MMIO-output transcript for switch word 0xE008; historical/final according to directory, not a new FPGA run.
reports/uart_e008.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/uart_e009.log	Stored firmware MMIO-output transcript for switch word 0xE009; historical/final according to directory, not a new FPGA run.

Exact file	Role and interpretation
reports/uart_e009.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/uart_e00a.log	Stored firmware MMIO-output transcript for switch word 0xE00A; historical/final according to directory, not a new FPGA run.
reports/uart_e00a.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/uart_e00b.log	Stored firmware MMIO-output transcript for switch word 0xE00B; historical/final according to directory, not a new FPGA run.
reports/uart_e00b.log.trace	Simulation event/retirement trace companion; not a physical-board capture. Version indicated by enclosing directory.
reports/user_routed_timing_report.rpt	Imported earlier user Vivado report/log; not the latest 40 MHz console.
reports/user_vivado_build_log.txt	Imported earlier user Vivado report/log; not the latest 40 MHz console.
reports/yosys_board.log	Historical Yosys structural/synthesis output; inspect completion/errors, not FPGA timing proof.
reports/yosys_check.log	Historical Yosys structural/synthesis output; inspect completion/errors, not FPGA timing proof.
reports/yosys_core_build_review.log	Historical Yosys structural/synthesis output; inspect completion/errors, not FPGA timing proof.
reports/yosys_timing_revision.log	Historical Yosys structural/synthesis output; inspect completion/errors, not FPGA timing proof.
reports/yosys_trace.log	Historical Yosys structural/synthesis output; inspect completion/errors, not FPGA timing proof.
reports/yosys_trace_build_review.log	Historical Yosys structural/synthesis output; inspect completion/errors, not FPGA timing proof.

21. Audited revision: changes and fresh validation

The revised source package fixes arithmetic right-shift signedness in alu_zbb.v, qualifies Zbb activity in pipeline_5stage.v and requires aggregate stall for the D-cache wait label in uart_trace_transport.v. dashboard.py now validates 57 unique host-known checks, all 100 packet indices, raw inputs and integer results, maintains diagnostic/flight EMA state, clears stale readouts and prevents synthetic green verification. Tests include a deterministic Python-generated ALU bench, tests/tb_zbb_observer.v, eight trace cases and 20 headless dashboard cases. No cache/FPU latency, firmware image or timing exception was changed.

After all fixes, the full regression passed, including 1,920 ALU cases, 24 observer checks and 20 dashboard tests. Fresh full-board functional simulations at both 40 and 50 MHz each completed 57 self-checks and 100 CSV packets with zero host failures. Board tests also check trace delivery, clock-gated completion and simulated LED0. The 15 existing mode/seed transcripts pass the stricter parser; these are not 15 newly rerun mode simulations. Physical GUI and FPGA operation remain untested.

Evidence files added: reports/viva_audit_regression.log (pre-fix baseline), reports/viva_audit_fixed_regression.log (post-fix full regression), reports/viva_audit_board_results.log (fresh post-fix 40/50 MHz board results), reports/viva_audit_host_probes.txt (historical pre-fix host gaps), reports/viva_audit_strict_host_results.log (15 stored transcripts with stricter parser), reports/viva_audit_alu_before.txt and reports/viva_audit_alu_after.txt (85 failures before, zero after), and reports/VIVA_AUDIT_REVISION.md (change summary). The updated packager regenerates MANIFEST_SHA256.json and reports/rtl_changes.patch. docs/Processor_Architecture_Audit_and_Viva_Guide.md and the PDF are the educational deliverables.

Required next hardware build: from the newly extracted audited project root, run vivado -mode batch -source board/build_vivado.tcl -tclargs 40. Inspect the new timing/DRC reports, then generate the bitstream if they pass. Do not program the previous bitstream and assume it contains this fix. RTL simulation at 50 MHz does not imply that the revised design meets 50 MHz physical timing.

No FPGA is available to this review session, and Vivado was not run locally. The guide provides reproducible evidence and an explicit remaining-issue list, not a guarantee of zero bugs.